Learning representations by back-propagating errors

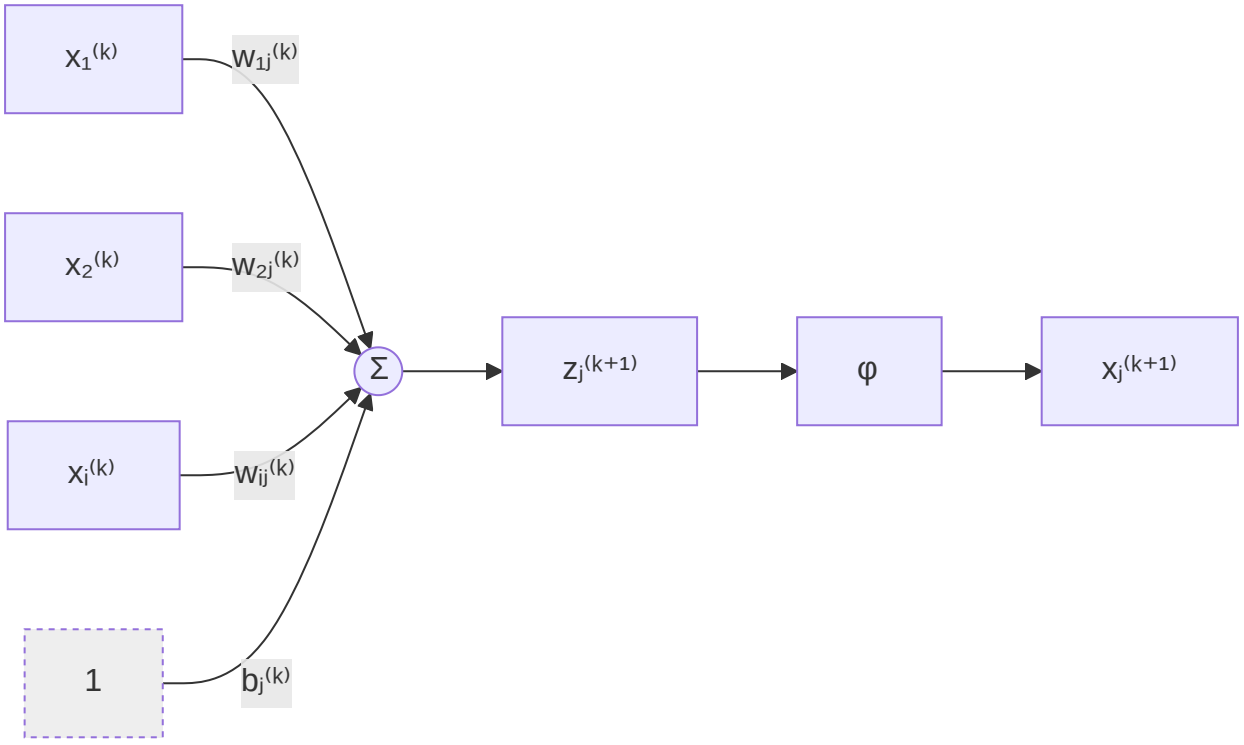
- Repeatedly adjusting the weights of the connections in a network so as to minimize a measure of the difference between the output vector and the ground truth vector.

Notation

- Layers are indexed $k = 0, 1, 2, \ldots, L$. Layer 0 is the input layer and layer L is the output layer.
- $x_i^{(k)}$ is the **output** (activation) of unit i in layer k. The network's input is $x^{(0)}$.
- $z_j^{(k+1)}$ is the input (pre-activation) to unit j of layer $k+1$.
- $w_{ij}^{(k)}$ is the weight on the connection running from unit i of layer k to unit j of layer $k+1$. $b_j^{(k)}$ is the bias of the unit j of layer $k+1$.
- The predicted output is the activation of the last layer, $x_j^{(L)} \equiv \hat{y}_j$, and y_j the ground truth vector.

Forward pass

- The input $z_j^{(k+1)}$ to a neuron j of the $(k+1)^{th}$ layer is a linear function of the outputs of the previous layer k as given below $z_j^{(k+1)} = \sum_i w_{ij}^{(k)} x_i^{(k)}$
- Units can be given biases by introducing an extra input to each unit called bias. Writing its weight seperately gives $z_j^{(k+1)} = \sum_i w_{ij}^{(k)} x_i^{(k)} + b_j^{(k)}$.
- A unit has a non-linear output $\phi(z_j^{(k+1)})$.
- Here is how the network looks like for one unit.



- ϕ (^activation) should be non-linear. Without it a stack of layers collapses into a single linear map and depth buys nothing.
- However, the use of a linear function before applying the non-linearity greatly simplifies the learning procedure.
- The forward pass is the computation of the states of each unit in a layer based on the inputs the receive from the previous layer using ^preactivation and ^activation starting at $x^{(0)}$ and ending at $x^{(L)}$.